

Halo! Saya telah menganalisis struktur dan kode dari proyek MyFinance Anda. Sistem ini memang sudah memiliki dasar yang baik dan terstruktur dengan rapi menggunakan pembagian rute (Blueprint) dan utilitas terpisah. Namun, agar aplikasi menjadi lebih stabil, efisien, dan mudah dirawat tanpa menambahkan fitur baru, berikut adalah daftar rekomendasi upgrade teknis yang siap dibaca:

1. Stabilitas & Keamanan Data (Stability)

- Aktifkan SQLite Foreign Key Constraints (PRAGMA foreign_keys = ON)
 - Masalah saat ini: SQLite secara default menonaktifkan pemeriksaan Foreign Key. Jika Anda menghapus sebuah Akun atau Kategori yang masih memiliki transaksi terkait, data transaksi tersebut akan menjadi yatim piatu (orphan). Hal ini akan memicu error fatal `AttributeError: 'NoneType' object has no attribute 'nama'` saat Jinja2 merender `trx.account.nama` di halaman transaksi.
 - Solusi: Daftarkan event listener di `app/__init__.py` agar otomatis mengaktifkan foreign keys setiap kali koneksi database dibuka:

```
python
from sqlalchemy import event
from sqlalchemy.engine import Engine
@event.listens_for(Engine, "connect")
def set_sqlite_pragma(dbapi_connection, connection_record):
    cursor = dbapi_connection.cursor()
    cursor.execute("PRAGMA foreign_keys=ON")
    cursor.close()
```

- Terapkan Proteksi Transaksi Database (try-except & rollback)
 - Masalah saat ini: Rute CRUD seperti `/tambah`, `/edit`, dan `/hapus` langsung mengeksekusi `db.session.commit()` tanpa penanganan error. Jika terjadi kegagalan (misalnya `IntegrityError` akibat batasan foreign key di atas), aplikasi akan menampilkan halaman error HTTP 500 dan sesi database Flask akan terkunci/rusak untuk request berikutnya.

- Solusi: Bungkus operasi commit dengan blok try-except di semua rute database (seperti pada `app/routes/akun.py` dan `app/routes/dropdown.py`), lalu panggil `db.session.rollback()` jika terjadi kegagalan:

```
python
try:
    db.session.commit()
except Exception as e:
    db.session.rollback()
    # Log error dan beri tahu user dengan flash message
```

- Validasi Input Sisi Server (Server-Side Validation)
 - Masalah saat ini: Rute `app/routes/transaksi.py` langsung mengonversi nominal menjadi Decimal tanpa validasi. Jika ada input nominal bernilai negatif, operasi matematika saldo (`akun.balance -= amount`) akan berubah menjadi penambahan saldo (bug logika).
 - Solusi: Tambahkan validasi dasar di backend sebelum menyimpan data: Pastikan nominal transaksi > 0 . Pastikan objek Akun dan Kategori yang dikirim benar-benar ada di database (tidak None).
- Aktifkan Mode WAL (Write-Ahead Logging) di SQLite
 - Masalah saat ini: Karena Flask berjalan secara multithreaded, Anda dapat mengalami masalah database is locked jika ada operasi penulisan data yang bertabrakan dengan pembacaan data.
 - Solusi: Jalankan perintah `PRAGMA journal_mode=WAL;` pada database SQLite Anda untuk memungkinkan pembacaan (read) berjalan secara paralel tanpa diblokir oleh penulisan (write).

2. Efisiensi & Kinerja (Efficiency)

- Perbaiki Bug Agregasi Laporan Pengeluaran (Koreksi Data)
 - Masalah saat ini: Fungsi pembantu `ambil_data_agregasi()` di `app/routes/laporan.py` (yang menghasilkan data CSV/PDF) tidak menyertakan filter `.filter(Category.type == 'expense')` seperti yang ada di

halaman web /laporan_hasil. Akibatnya, data pemasukan (income) ikut terhitung ke dalam total pengeluaran di laporan PDF/CSV yang diunduh.

- Solusi: Tambahkan filter kategori pengeluaran pada query agregasi agar data laporan PDF/CSV sinkron dan akurat dengan apa yang tampil di halaman web.
- Optimalkan Script Otomatisasi (clean_web.bat & clean_python.bat)
 - Masalah saat ini: Kedua file batch ini selalu menjalankan npm install dan pip install setiap kali dijalankan. Hal ini memperlambat eksekusi skrip secara signifikan, memakan bandwidth internet, dan menulis ke disk secara berulang tanpa kebutuhan mendesak.
 - Solusi: Buat file package.json di root direktori untuk menyimpan daftar linter (eslint, prettier, stylelint). Pada batch script, lakukan pengecekan keberadaan folder node_modules atau folder .venv terlebih dahulu. Jalankan instalasi hanya jika dependensi tersebut belum terpasang.

3. Kemudahan Perawatan (Maintainability)

- Integrasikan Fitur Flash Message di Template Utama
 - Masalah saat ini: Belum ada mekanisme untuk menampilkan pesan sukses atau pesan peringatan gagal (seperti jika akun gagal dihapus karena masih memiliki relasi transaksi) di antarmuka web.
 - Solusi: Gunakan fitur bawaan Flask get_flashed_messages(). Cukup tambahkan blok berikut di bagian atas penampung .main-content pada app/templates/base.html:

```
html
{% with messages = get_flashed_messages(with_categories=true) %}
{% if messages %}
  <div class="flash-messages">
    {% for category, message in messages %}
      <div class="flash-message alert-{{ category }}">{{ message }}</div>
    {% endfor %}
  </div>
{% endif %}
```

```
{% endwith %}
```

Kemudian atur desain visual notifikasinya di `style.css`.

- Tangani Peringatan Depresiasi Fungsi Waktu (DeprecationWarning)
 - Masalah saat ini: Fungsi `datetime.utcnow()` yang digunakan di beberapa file telah didepresiasi sejak Python 3.12 dan dapat memicu peringatan log.
 - Solusi: Ganti `datetime.utcnow()` dengan `datetime.now(timezone.utc)` (menggunakan `from datetime import timezone`) agar kode siap untuk versi Python terbaru.
- Satukan Query Agregasi untuk Menghindari Duplikasi
 - Masalah saat ini: Query data pengeluaran ditulis dua kali di `app/routes/laporan.py` — sekali di rute `laporan_hasil()` dan sekali lagi di fungsi pembantu `ambil_data_agregasi()`.
 - Solusi: Refaktor rute tersebut agar `laporan_hasil()` memanggil fungsi pembantu `ambil_data_agregasi()` secara langsung. Hal ini mengurangi risiko terjadinya perbedaan kode (code drift) di masa mendatang.

Ringkasan Pengerjaan

Analisis Struktur: Mengulas rute aplikasi Flask, model basis data, template frontend, serta skrip otomatisasi Batch.

Identifikasi Masalah: Menemukan kerentanan hilangnya integritas referensial SQLite, potensi crash (HTTP 500) saat kegagalan penghapusan akun/kategori, inkonsistensi data pada laporan PDF/CSV, dan inefisiensi pengunduhan dependensi di skrip batch. **Penyusunan Solusi:** Merangkum rekomendasi perbaikan spesifik berbasis kode agar sistem MyFinance Anda semakin kokoh dan efisien untuk digunakan sehari-hari.